

```
# MASTER AUDIT & REFACTOR REQUEST: ENGINE_1 COINGLASS
TRADING ENGINE
```

1. Context & Architecture Overview

'Engine_1.py' is a real-time cryptocurrency algorithmic trading engine written in Python (Asyncio + Playwright + LightGBM/XGBoost/CatBoost + MetaTrader 5).

The engine operates 6 distinct strategies across 18 crypto symbols (BTC, ETH, XRP, SOL, BNB, DOGE, ADA, TRX, LINK, AVAX, XLM, HBAR, LTC, DOT, XMR, NEAR, XAU, NATGAS):

- 'S1_Liquidation' (Liquidation spillover)
- 'S2_CVD' (Cumulative Volume Delta divergence)
- 'S3_Trend' (Trend continuation & pullback)
- 'ML_Vwap_Reversal' (VWAP mean-reversion ML model)
- 'S5_Microstructure' (Orderbook imbalance & bid/ask delta)
- 'S6_SMC_Orderflow' (Smart Money Concepts orderflow & footprint)

2. Chronological Log of System Failures & Symptoms

During live diagnostic runs, the following critical failures were observed:

- * "Disappearing TradingView Indicators & Continuous Re-injection Loops"
- Indicators injected into Coinglass TradingView iframe legends periodically vanish.
- The engine's watchdog detects missing indicators and triggers 'inject_and_configure_all'.
- A race condition existed where 'TradingViewApi.changeSymbol()' or 'chartWidgetCollection.setResolution(15)' triggered an asynchronous chart reload "during" indicator injection, instantly wiping newly injected studies and causing an infinite re-injection loop.

2. **"Event Loop Latency & Binance Feed Dropouts:"**

- Warnings observed: **[!ALERT] [LATENCY] Event loop blocked for 0.91s!**
Potential CPU-bound $\downarrow$ $\uparrow$ η event loop.
- Followed by: **[Binance Feed] [WARN] Connection issues detected (all queries failed)**.
- Cause: 16 threads in **'ThreadPoolExecutor'** running Pandas feature calculations (`._run_inference_with_copied_history`) on every tick simultaneously, causing Global Interpreter Lock (GIL) starvation on the main asyncio event loop, which caused 'aiohttp' REST requests to time out.

3. ****Chrome Page Refreshes & Memory Pressure:****
- Headless Chrome running 18 Cointelligence chart tabs consumes excessive RAM, leading Chrome to discard or refresh background tabs, causing complete loss of DOM indicators and frame detachment errors.

- Despite price streams running, zero trades were placed.
- Gaps identified: Pre-warmup requirements ('len(history) < 250' candles), overly strict strategy gating thresholds (e.g., zero-variance z-score normalizations), missing symbol mappings for MT5 brokers ('BTCUSD' vs 'BTCUSD'), and silent drops when 'dry_run' flags or cooldown locks ('trade_tracker') remain engaged.

3. Comprehensive Audit Scope & Required Deliverables

Please conduct an intensive, zero-slop audit of 'Engine_1.py', 'live_unified_predictor.py', and 'mt5_broker.py' covering the following 5 areas:

Area A: Chrome Tab Stability & Indicator Persistence

- How can we permanently stop CoinGlass/TradingView from dropping indicators without relying on high-frequency DOM polling?
- Can we inject a pre-configured chart layout directly into 'window.localStorage' before DOM instantiation to make indicator loading native?
- What exact Chrome browser args should be passed to Playwright to disable tab discarding, background throttling, and renderer memory limits?

- **### Area B: Event Loop Protection & GIL Isolation**
- Audit all synchronous operations called inside the "asyncio" event loop (e.g., `SnapshotStore.update()`, `trade_tracker.check_exits()`, MT5 API calls, Pandas manipulations).
- Is `ThreadPoolExecutor` sufficient, or should heavy Pandas feature calculations (`'prep_alpha'`, `'prep_trend'`, `'prep_vwap'`) be offloaded to a `'ProcessPoolExecutor'` or process worker queue?
- Provide exact code changes to ensure tick processing and REST polling never block the event loop for > 10ms.

Area C: Trade Execution Path Verification (Gate-by-Gate Audit)

Trace the full execution path from tick ingestion to `mt5.order_send`:

```

1  **Warmup Gate:** Ensure candle history backfills correctly so symbols
2  reach the 250-candle threshold immediately on startup without waiting 60+
3  hours.
4
5  **Strategy Gate:** Audit all mathematical formulas in
6  twe_unified_predictor.py for edge cases (e.g. division by zero, NaN
7  handling, sign mismatches in directional filters).
8
9  **Probability Gate:** Audit model ensemble scoring ('p_long' / 'p_short'
10 thresholds vs manifest outliers).
11
12 **Governor Gate:** Audit daily max drawdown, maximum concurrent
13 trades per symbol, and trade cooldown times (e.g., TP/SL locks) to ensure
14 legitimate setups aren't permanently blocked.
15
16 **Broker Bridge Gate:** Audit order price normalization
17 ('translate_to_mf5_price', lot size calculation, symbol translation
18 ('BTCUSD' to 'BTCUSD_A' or error handling on MT5 order rejection.

```

Area D: OOS Backtest vs Live Parity

- Compare live feature calculation in 'live_unified_predictor.py' against standalone OOS backtest scripts ('opt_s1_colab_standalone.py' through 'opt_s6_colab_standalone.py').
- Flag any discrepancies in lookback windows, EMA calculation methods ('adjust=False' vs star ↓), fill price assumptions (open of next bar vs close of current bar), and fee/slippage subtractions.

4. Deliverable Format

1. ****Architectural Diagnosis:**** High-level summary of structural design flaws.
2. ****Direct Refactored Code Snippets:**** Ready-to-apply Python/JS code blocks targeting `Engine_1.py`, `live_unified_predictor.py`, and `mt5_broker.py`.
3. ****Verification Checklist:**** Actionable steps to test and verify trade triggers under dry-run and live conditions.

The engine operates 6 distinct strategies across 18 crypto symbols (BTC, ETH, XRP, SOL, BNB, DOGE, ADA, TRX, LINK, AVAX, XLM, HBAR, LTC, DOT, XMR, NEAR, XAU, NATGAS):

- 'S1_Liquidation' (Liquidity on spillover)
- 'S2_CVD' (Cumulative Volume Delta divergence)
- 'S3_Trend' (Trend continuation & pullback)
- 'ML_Vwap_Reversal' (VWAP mean-reversion ML model)
- 'S5_Microstructure' (Orderbook imbalance & bid/ask delta)
- 'S6_SMC_Orderflow' (Smart Money Concepts orderflow & footprint)

2. Chronological Log of System Failures & Symptoms
During live diagnostic runs, the following critical failures were observed:

1. "Disappearing TradingView Indicators & Continuous Re-injection Loops."
 - Indicators injected into Coinglass TradingView iframe legends periodically vanish.
 - The engine's watchdog detects missing indicators and triggers 'inject_and_configure_all()'.
 - A race condition existed where 'tradingViewApi.changeSymbol()' or 'chartWidgetCollection.setResolution(15)' triggered an asynchronous chart reload "during" indicator injection, instantly wiping newly injected studies and causing an infinite re-injection loop.
2. "Event Loop Latency & Binance Feed Dropouts."
 - Warnings observed: '[ALERT][LATENCY] Event loop blocked for 0.91s! Potential CPU-bound task in event loop.'
 - Followed by: '[Binance Feed][WARN] Connection issues detected (all queries failed)'.
 - Cause: 16 threads in 'ThreadPoolExecutor' running Pandas feature calculations ('_run_inference_with_copied_history') on every tick simultaneously, causing Global Interpreter Lock (GIL) starvation on the main asyncio event loop, which caused 'aiohttp' REST requests to time out.
3. "Chrome Page Refreshes & Memory Pressure."
 - Headless Chrome running 18 Coinglass chart tabs consumes excessive RAM, leading Chrome to discard or refresh background tabs, causing complete loss of DOM indicators and frame detachment errors.
4. "Zero-Trade Execution Deficit."
 - Despite price streams running, zero trades were placed.
 - Gaps identified: Pre-warmup requirements ('len(history) < 250' candles), overly strict strategy gating thresholds (e.g., zero-variance z-score normalizations), missing symbol mappings for MT5 brokers ('BTCUSD' vs 'BTCUSD'), and silent drops when 'dry_run' flags or cooldown locks ('trade_tracker') remain engaged.

3. Comprehensive Audit Scope & Required Deliverables

Please conduct an intensive, zero-slop audit of 'Engine_1.py', 'live_unified_predictor.py', and 'mt5_broker.py' covering the following 5 areas:

Area A: Chrome Tab Stability & Indicator Persistence

- How can we permanently stop Coinglass/TradingView from dropping indicators without relying on high-frequency DOM polling?
- Can we inject a pre-configured chart layout directly into 'window.localStorage' before DOM instantiation to make indicator loading native?
- What exact Chrome browser args should be passed to Playwright to disable tab discarding, background throttling, and renderer memory limits?

Area B: Event Loop Protection & GIL Isolation

- Audit all synchronous operations called inside the 'asyncio' event loop (e.g., 'SnapshotStore.update()', 'trade_tracker.check_exits()', MT5 API calls, Pandas manipulations).
- Is 'ThreadPoolExecutor' sufficient, or should heavy Pandas feature calculations ('prep_alpha', 'prep_trend', 'prep_vwap') be offloaded to a 'ProcessPoolExecutor' or process worker queue?
- Provide exact code changes to ensure tick processing and REST polling never block the event loop for > 10ms.

Area C: Trade Execution Path Verification (Gate-by-Gate Audit)

Trace the full execution path from tick ingestion to 'mt5_order_send':

1. "Warmup Gate:" Ensure candle history backfills correctly so symbols reach the 250-candle threshold immediately on startup without waiting 60+ hours.
2. "Strategy Gate:" Audit all mathematical formulas in 'live_unified_predictor.py' for edge cases (e.g., division by zero, NaN propagation, sign mismatches in directional filters).
3. "Probability Gate:" Audit model ensemble scoring ('p_long' / 'p_short' thresholds vs manifest cutoffs).
4. "Governor Gate:" Audit daily max drawdown, maximum concurrent trades per symbol, and trade cooldown timers (e.g., TP/SL locks) to ensure legitimate setups aren't permanently blocked.
5. "Broker Bridge Gate:" Verify price normalization ('_translate_to_mt5_price'), lot size calculation, symbol translation ('BTCUSD' -> 'BTCUSD.a'), and error handling on MT5 order rejection.

Area D: OOS Backtest vs Live Parity

- Compare live feature calculation in 'live_unified_predictor.py' against standalone OOS backtest scripts ('opt_s1_colab_standalone.py' through 'opt_s6_colab_standalone.py').
- Flag any discrepancies in lookback windows, EMA calculation methods ('adjust=False' vs standard), fill price assumptions (open of next bar vs close of current bar), and fees/slippage subtractions.

4. Deliverable Format

1. "Architectural Diagnosis:" High-level summary of structural design flaws.
2. "Direct Refactored Code Snippets:" Ready-to-apply Python/JS code blocks targeting 'Engine_1.py', 'live_unified_predictor.py', and 'mt5_broker.py'.
3. "Verification Checklist:" Actionable steps to test and verify trade triggers under dry-run and live conditions.

Below is the audit I would apply first. The short version: **your zero-trade + latency problem is not one bug; it is a coupled architecture issue:** TradingView DOM is being treated as a live data bus, ML is being called too often from a GIL-bound thread pool, and several downstream gates silently reject trades.

1. Architectural Diagnosis

Critical structural flaws

1. **UI/DOM is in the live data path**
 - Coinglass/TradingView indicators are not stable enough to be a critical runtime dependency.
 - The live engine should survive even if every TradingView study disappears.
 - Use TradingView only for seeding/visual fallback; use network/API feeds for live values.
2. **ML inference is tick-triggered instead of bar-close-triggered**
 - SnapshotStore.update() fires predictor tasks on every price update.
 - UnifiedLivePredictor then runs Pandas/feature logic up to once per second per symbol.
 - With 18 symbols, this is enough to starve the asyncio event loop even though it is "in threads".
3. **ThreadPoolExecutor is not sufficient for heavy Pandas**
 - Pandas frequently holds the GIL.
 - 16 ML threads can block the main event loop.
 - Immediate hotfix: reduce ML workers to 1-2 and infer only on closed 15m bars.
 - Proper fix: one process worker for ML feature generation/inference.
4. **Live feature parity is currently unsafe**

- In `live_unified_predictor.py`, important training columns are added **after** `generate_features_standard()` / `prepare_data()`.
 - That means rolling features may be computed without POC Price, RSI, Bid USD, Ask USD, Whale Ind, etc.
 - This can make live probabilities meaningless versus OOS.
5. **Governor gates can silently reject nearly everything**
- `trigger_entry()` rejects `stop_pct < 0.003`.
 - `UnifiedLivePredictor` uses `s1_mult * atr` directly.
 - If 15m ATR is less than 0.3%, trades are rejected instead of widened.
 - Also `stop_dist > 5 * ATR` conflicts with the 0.3% minimum stop.
6. **MT5 bridge has pending-order and symbol-map weaknesses**
- Pending order success retcode can be `TRADE_RETCODE_PLACED`, not only `DONE`.
 - `_map_symbol()` needs explicit broker suffix support.
 - `BTCUSDOT` -> `BTCUSD` is often insufficient; many brokers use `BTCUSD.a`, `BTCUSDm`, `BTCUSD#`, etc.
7. **Hardcoded live mode bug**
- `MT5_LIVE = False` means real orders are permanently disabled regardless of env.

2. Direct Refactored Code Snippets

A. Engine_1.py patches

A1. Replace executor and MT5 live config

Replace:

```
ML_EXECUTOR = ThreadPoolExecutor(max_workers=1, thread_name_prefix="MLPredictors")
```

With:

```
ML_EXECUTOR = ThreadPoolExecutor(
    max_workers=int(os.environ.get("ML_THREADS", "2")),
    thread_name_prefix="MLPredictors"
)

RISK_EXECUTOR = ThreadPoolExecutor(
    max_workers=1,
    thread_name_prefix="RiskEngine"
)
```

Replace:

```
MT5_LIVE = False
```

With:

```
MT5_LIVE = (
    os.environ.get("MT5_LIVE", "0").strip().lower() in {"1", "true", "yes", "live"}
    and EXECUTION_MODE.upper() == "LIVE"
)
```

A2. Fix wrong Binance historical endpoints

Replace:

```
url = f"https://fapi.binance.com/fapi/v1/openInterestHist?symbol={symbol}&period=15m&limit=120"
```

With:

```
url = f"https://fapi.binance.com/futures/data/openInterestHist?symbol={symbol}&period=15m&limit=500"
```

Replace:

```
url = f"https://fapi.binance.com/data/globalLongShortAccountRatio?symbol={symbol}&period=15m&limit=120"
```

With:

```
url = f"https://fapi.binance.com/futures/data/globalLongShortAccountRatio?symbol={symbol}&period=15m&limit=500"
```

This directly affects warmup quality for oi and ls_ratio.

A3. Stop synchronous risk/ML work inside the `SnapshotStore.update()` lock

Replace your current `SnapshotStore.update()` with this pattern.

```
class SnapshotStore:
    def __init__(
        self,
        symbols: List[str],
        liquidation_predictor: Any = None,
        trade_tracker: Any = None,
        trend_pull_predictor: Any = None,
    ):
        self.data: Dict[str, AssetSnapshot] = {s: AssetSnapshot(symbol=s) for s in symbols}
        self.locks = {s: asyncio.Lock() for s in symbols}
        self_seq = 0

        self.predictor = None
        self.liquidation_predictor = liquidation_predictor
        self.trend_pull_predictor = trend_pull_predictor
        self.trade_tracker = trade_tracker

        self.ml_tasks: Dict[str, asyncio.Future] = {}
        self.risk_pending: set[str] = set()
        self.last_ml_submit_ns: Dict[str, int] = {}

    def _build_atr_dict(self, symbol: str) -> dict:
        atr_dict = {}

        if self.predictor:
            try:
                with self.predictor._lock:
                    cached = self.predictor._cached_signal.get(symbol, {})
                    atr_dict[STRATEGY_DISPLAY_NAME] = cached.get("atr_val", 0.0)
            except Exception:
                pass
```

```

        if getattr(self, "liquidation_predictor", None):
            unified_atr = getattr(self.liquidation_predictor, "latest_atr", {}).get(symbol,
0.0)

            for strat in [
                "S1_liquidation",
                "S2_OVD",
                "S3_trend",
                "ML_Wrap_Reversal",
                "S5_Microstructure",
                "S6_SMC_Orderflow",
                "ML_liquidation_Runner",
            ]:
                atr_dict[strat] = unified_atr

            return atr_dict

def submit_risk_update(self, symbol: str, price: float) -> None:
    if not self.trade_tracker:
        return

    if symbol in self._risk_pending:
        return

    self._risk_pending.add(symbol)

def _risk_job():
    try:
        atr_dict = self._build_atr_dict(symbol)
        self.trade_tracker.check_exits(symbol, price, atr_dict)
        self.trade_tracker.update_live_pnl(symbol, price, self)
    finally:
        self._risk_pending.discard(symbol)

    try:
        RISK_EXECUTOR.submit(_risk_job)
    except Exception as exc:
        self._risk_pending.discard(symbol)
        print(f"[RiskExecutor] submit failed for {symbol}: {exc}")

def _submit_ml_update(self, symbol: str, snap: AssetSnapshot) -> None:
    now_ns = time.time_ns()

    # Keep predictor candle state fresh, but avoid executor spam.
    # UnifiedLivePredictor should only do heavy inference on a new closed bar.
    last_ns = self._last_ml_submit_ns.get(symbol, 0)
    if now_ns - last_ns < 250_000_000: # 250 ms per symbol
        return
    self._last_ml_submit_ns[symbol] = now_ns

    prev_task = self._ml_tasks.get(symbol)
    if prev_task and not prev_task.done():
        if DIAG:
            DIAG.record(
                symbol=symbol,
                strategy="All",
                bar_ts=int(time.time() // 900) * 900,
                strat_triggered=False,
                p_long=0.0,
                p_short=0.0,
                threshold=0.0,
                skipped_backlog=True,
                skipped_duplicate=False,
                skipped_cooldown=False,
                trade_sent=False,
            )
        return

def _run_ml_predictors(sym: str, snap: AssetSnapshot, tracker):
    if self.predictor:
        self.predictor.on_tick_update(sym, snap_obj, tracker)

    if (
        getattr(self, "liquidation_predictor", None)
        and self.liquidation_predictor is not self.predictor
    ):
        self.liquidation_predictor.on_tick_update(sym, snap_obj, tracker)

    loop = asyncio.get_running_loop()
    task = loop.run_in_executor(
        ML_EXECUTOR,
        _run_ml_predictors,
        symbol,
        snap,
        self.trade_tracker,
    )

def _on_done(f):
    try:
        f.result()

        ml_sigs = {}
        armed_str = ""

        if self.predictor:
            with self.predictor._lock:
                cached = self.predictor._cached_signal.get(symbol, {})
                ml_sigs.update(cached.get("ml_signals", {}))
                armed_str = cached.get("armed_str", armed_str)

        if getattr(self, "liquidation_predictor", None):
            with self.liquidation_predictor._lock:
                cached = self.liquidation_predictor._cached_signal.get(symbol, {})
                ml_sigs.update(cached.get("ml_signals", {}))
                armed_str = cached.get("armed_str", armed_str)

        if getattr(self, "trend_pull_predictor", None):
            with self.trend_pull_predictor._lock:
                cached = self.trend_pull_predictor._cached_signal.get(symbol, {})
                ml_sigs.update(cached.get("ml_signals", {}))
                armed_str = cached.get("armed_str", armed_str)

        cur_snap = self._data.get(symbol)
        if cur_snap:
            self._data[symbol] = dataclasses.replace(
                cur_snap,
                ml_signals=dict(ml_sigs),
                strategy_armed=armed_str,
            )

    except Exception as e:
        print(f"[ML_EXECUTOR] Error in predictors for {symbol}: {e}")

    task.add_done_callback(_on_done)
    self._ml_tasks[symbol] = task

async def update(self, symbol: str, source: str = "binance", **patch: Any) -> None:
    if symbol not in self._data:
        return

    new_snap = None
    price_fresh = False

    async with self._locks[symbol]:
        cur = self._data[symbol]
        clean_patch = {}

        for k, v in patch.items():
            if not hasattr(cur, k):
                continue

```

```

        if k == "price":
            fv = finite_float_or_none(v)
            if fv is None or fv <= 0.0:
                continue
            clean_patch[k] = fv

        elif k in _FLOAT_FIELDS:
            fv = finite_float_or_none(v)
            if fv is None:
                continue
            clean_patch[k] = fv

        else:
            clean_patch[k] = v

    if not clean_patch:
        return

    self._seq += 1
    new_snap = dataclasses.replace(
        cur,
        seq=self._seq,
        ts_ns=time.time_ns(),
        **clean_patch,
    )

    self._data[symbol] = new_snap
    price_fresh = "price" in clean_patch and new_snap.price > 0.0

    # Important: risk + PL submitted outside asyncio lock.
    if price_fresh and new_snap:
        self._submit_risk_update(symbol, new_snap.price)
        self._submit_pl_update(symbol, new_snap)

def snapshot(self) -> Dict[str, AssetSnapshot]:
    return dict(self._data)

```

Copy

A4. Chrome stability args

Replace your Playwright launch args with:

```

CHROME_STABILITY_ARGS = [
    "--start-maximized",
    "--window-size=1920,1080",

    # Prevent background throttling / occlusion throttling.
    "--disable-background-timer-throttling",
    "--disable-backgrounding-occluded-windows",
    "--disable-renderer-backgrounding",
    "--disable-features=CalculateNativeMinOcclusion,IntensiveWakeUpThrottling,TabFreeze,BackForwardCache",

    # Reduce extra browser services.
    "--disable-extensions",
    "--disable-component-update",
    "--disable-sync",
    "--no-first-run",
    "--no-default-browser-check",
    "--metrics-recording-only",
    "--mute-audio",

    # Gives TradingView scripts more V8 heap headroom.
    "--js-flags=--max-old-space-size=4096",
]

```

Copy

Then launch with:

```

user_data_dir = os.path.join(base_dir, "chrome_profile")

ctx = await pw.chromium.launch_persistent_context(
    user_data_dir,
    headless=False,
    viewport={"width": 1920, "height": 1080},
    args=CHROME_STABILITY_ARGS,
)

```

Copy

Also block unnecessary heavy assets:

```

async def block_heavy_assets(route):
    req = route.request
    if req.resource_type in ("image", "media", "font"):
        await route.abort()
    else:
        await route.continue_()

await ctx.route("**/*", block_heavy_assets)

```

Copy

A5. Do not inject both tabs concurrently

Replace:

```

await asyncio.gather(
    tab1.inject_and_configure_all(focus_lock),
    tab2.inject_and_configure_all(focus_lock)
)

```

Copy

with:

```

await tab1.inject_and_configure_all(focus_lock)
await tab2.inject_and_configure_all(focus_lock)

```

Copy

Inside `CoinglassTab.inject_and_configure_all()`, wrap the body:

```

async def inject_and_configure_all(self, focus_lock: asyncio.Lock, symbols_only: bool = False):
    async with focus_lock:
        return await self._inject_and_configure_all_locked(focus_lock, symbols_only)

```

Copy

Then rename your existing body to:

```

async def _inject_and_configure_all_locked(
    self,
    focus_lock: asyncio.Lock,
    symbols_only: bool = False,
):
    ...

```

Copy

This prevents simultaneous TradingView reload/injection races.

B. Coinglass / TradingView indicator persistence

Important answer on localStorage

Can you inject a preconfigured chart layout into window.localStorage?

Technically yes, but it is not reliable as the main fix.

Reasons:

1. The TradingView chart is inside iframes.
2. The iframe may have a different origin/storage namespace.
3. Coinglass layout /tv/layout/s9 is likely server-layout-backed.
4. TradingView's layout schema is private and can change.
5. Injecting wrong localStorage keys can silently corrupt the layout.

Use this only as an optimization after discovery.

Discovery snippet:

```
async def dump_tv_storage(frame, out_path: str):
    data = await frame.evaluate("""
    () => {
        const out = {};
        for (const k of Object.keys(localStorage)) {
            if (/chart|layout|study|trading|tv/i.test(k)) {
                out[k] = localStorage.getItem(k)?.slice(0, 5000);
            }
        }
        return out;
    }
    """)
    with open(out_path, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2)
```

Primary fix should be:

- Save a native Coinglass layout once.
- Stop changing symbols/resolution at runtime.
- Stop high-frequency DOM legend polling.
- Use network/API capture as the live data plane.
- Use DOM only as a low-frequency health fallback.

C. live_unified_predictor.py patches

C1. Add safe helpers near the top

```
try:
    from pipeline_diag import DIAG
except Exception:
    DIAG = None

MIN_LIVE_SL_PCT = float(os.environ.get("MIN_LIVE_SL_PCT", "0.003"))
MIN_WARMUP_BARS = int(os.environ.get("MIN_WARMUP_BARS", "250"))
```

Add inside UnifiedLivePredictor:

```
def _f(self, value, default: float = 0.0) -> float:
    try:
        v = float(value)
        return v if np.isfinite(v) else default
    except Exception:
        return default

def _num_col(self, df: pd.DataFrame, name: str, default: float = 0.0, absval: bool = False) -> pd.Series:
    if name not in df.columns:
        s = pd.Series(default, index=df.index, dtype="float64")
    else:
        s = pd.to_numeric(df[name], errors="coerce").replace([np.inf, -np.inf], np.nan).fillna(default)
    return s.abs() if absval else s

def _standardize_history_df(self, history: list[dict]) -> pd.DataFrame:
    df = pd.DataFrame(history).copy()
    if df.empty or "open_time" not in df.columns:
        return pd.DataFrame()

    ot = pd.to_numeric(df["open_time"], errors="coerce")
    ot = np.where(ot > 1e11, ot / 1000.0, ot)
    df["open_time"] = ot
    df = df.dropna(subset=["open_time"])
    df["open_time"] = df["open_time"].astype("int64")
    df = df.sort_values("open_time").drop_duplicates("open_time", keep="last").reset_index(drop=True)

    out = pd.DataFrame(index=df.index)
    out["open_time"] = df["open_time"]
    out["ts"] = pd.to_datetime(df["open_time"], unit="s", utc=True).dt.tz_localize(None)

    out["Open"] = self._num_col(df, "open")
    out["High"] = self._num_col(df, "high")
    out["Low"] = self._num_col(df, "low")
    out["Close"] = self._num_col(df, "close")
    out["Volume"] = self._num_col(df, "volume")

    out["CVD"] = self._num_col(df, "fut_cvd")
    out["Agg. OI"] = self._num_col(df, "oi")
    out["Agg. Funding Rate"] = self._num_col(df, "funding")
    out["Long/Short Ratio (Account)"] = self._num_col(df, "ls_ratio", 1.0)

    out["Agg. Liq Long"] = self._num_col(df, "liq_long", 0.0, absval=True)
    out["Agg. Liq Short"] = self._num_col(df, "liq_short", 0.0, absval=True)

    out["Bid Qty"] = self._num_col(df, "coins_bid", 0.0, absval=True)
    out["Ask Qty"] = self._num_col(df, "coins_ask", 0.0, absval=True)
    out["Buy Qty"] = out["Bid Qty"]
    out["Sell Qty"] = out["Ask Qty"]
    out["Delta Qty"] = out["Bid Qty"] - out["Ask Qty"]
    out["Candle Delta"] = out["Delta Qty"]
    out["total_qty"] = out["Bid Qty"] + out["Ask Qty"]

    out["Bid USD"] = self._num_col(df, "dollars_bid", 0.0, absval=True)
    out["Ask USD"] = self._num_col(df, "dollars_ask", 0.0, absval=True)
    out["Delta USD"] = out["Bid USD"] - out["Ask USD"]

    out["Bid Trades"] = self._num_col(df, "tk_buy_cnt", 0.0, absval=True)
    out["Ask Trades"] = self._num_col(df, "tk_sell_cnt", 0.0, absval=True)

    out["RSI"] = self._num_col(df, "rsi", 50.0)
    out["rsi"] = out["RSI"]
    out["whale Ind"] = self._num_col(df, "whale_idx", 0.0)

    poc = self._num_col(df, "fp_poc", np.nan)
    out["POC Price"] = np.where((poc <= 0.0) | np.isnan(poc), out["Close"], poc)

    out["Price Low"] = out["Low"]
    out["Price High"] = out["High"]
    out["Mid Price"] = (out["High"] + out["Low"]) / 2.0
    out["Net Shorts"] = 0.0
    out["Net Longs"] = 0.0
```

```

out["Candle"] = np.arange(len(out))

return out.replace([np.inf, -np.inf], np.nan).ffill().bfill().fillna(0.0)

def _make_btc_ref(self, symbol: str, fallback_df: pd.DataFrame) -> pd.DataFrame:
    btc_hist = []

    with self._locks:
        if "BTCUSD01" in self.candles_history:
            btc_hist = list(self.candles_history["BTCUSD01"])

    if btc_hist and len(btc_hist) >= 50:
        btc_df = self._standardize_history_df(btc_hist)
    else:
        btc_df = fallback_df.copy()

    ref = pd.DataFrame()
    ref["ts"] = btc_df["ts"]
    ref["btc_Close"] = btc_df["Close"]
    ref["btc_CVD"] = btc_df["CVD"]
    return ref.replace([np.inf, -np.inf], np.nan).ffill().bfill().fillna(0.0)

def _manifest_for(self, strat: str, symbol: str) -> dict:
    key = f"{strat}_{symbol}"
    data = self.manifest_data.get(key)
    if isinstance(data, dict):
        return data

    data = self.manifest_data.get(strat, {})
    if isinstance(data, dict):
        sym_data = data.get(symbol)
        if isinstance(sym_data, dict):
            return sym_data

    return {}

def _threshold_for(self, m_data: dict, direction: int) -> float:
    if direction == 1:
        for k in ("prob_threshold_long", "long_threshold", "threshold_long"):
            if k in m_data:
                return float(m_data[k])
    else:
        for k in ("prob_threshold_short", "short_threshold", "threshold_short"):
            if k in m_data:
                return float(m_data[k])

    return float(m_data.get("prob_threshold", m_data.get("confidence", 0.60)))

```

Copy

C2. Fix model hot-swap path

Replace `check_model_updates()` with:

```

def check_model_updates(self):
    now = time.time()
    if now - getattr(self, "_last_model_check_time", 0.0) < 10.0:
        return
    self._last_model_check_time = now

    manifest_paths = [
        os.path.join(BASE_DIR, "models", "manifest.json"),
        os.path.join(BASE_DIR, "liquidation", "models", "manifest.json"),
        os.path.join(BASE_DIR, "ml_trend_pull", "models", "manifest.json"),
    ]

    mtimes = [os.path.getmtime(p) for p in manifest_paths if os.path.exists(p)]
    if not mtimes:
        return

    newest = max(mtimes)
    if newest > self._last_model_mtime:
        with self._locks:
            if newest <= self._last_model_mtime:
                return
            self._last_model_mtime = newest

    print(f"[UnifiedPredictor] Detected updated manifest. Hot-swapping models...")
    self._load_models()

```

Copy

C3. Only infer on newly closed bars

Replace `on_tick_update()` with this version:

```

def on_tick_update(self, symbol, snap, trade_tracker=None):
    if snap.price <= 0.0:
        return snap

    self.check_model_updates()

    history_copy = None
    current_candle_copy = None
    should_trade_infer = False

    with self._locks:
        now = time.time()
        open_time = int(now // 900) * 900
        history = self.candles_history[symbol]

        if symbol not in self.current_candle or self.current_candle[symbol].get("open_time") != open_time:
            prev = self.current_candle.get(symbol)

            if prev and int(prev.get("open_time", 0)) < open_time:
                prev_ot = int(prev["open_time"])
                if not history or int(history[-1].get("open_time", 0)) != prev_ot:
                    history.append(dict(prev))

            self.current_candle[symbol] = {
                "open_time": open_time,
                "open": snap.price,
                "high": snap.price,
                "low": snap.price,
                "close": snap.price,
                "volume": snap.volume,
                "fut_cvd": snap.fut_cvd,
                "liq_long": abs(snap.liq_long),
                "liq_short": abs(snap.liq_short),
                "coins_bid": abs(snap.coins_bid),
                "coins_ask": abs(snap.coins_ask),
                "dollars_bid": abs(snap.dollars_bid),
                "dollars_ask": abs(snap.dollars_ask),
                "tk_buy_cnt": abs(snap.tk_buy_cnt),
                "tk_sell_cnt": abs(snap.tk_sell_cnt),
                "fp_poc": snap.fp_poc,
                "oi": snap.oi,
                "funding": snap.funding,
                "ls_ratio": snap.ls_ratio,
                "rsi": snap.rsi,
                "whale_idx": snap.whale_idx,
            }

        if len(history) >= MIN_WARMUP_BARS:
            last_closed = int(history[-1].get("open_time", 0))
            if last_closed > 0 and last_closed != self._last_predict_bar.get(symbol, 0):
                self._last_predict_bar[symbol] = last_closed
                history_copy = list(history)

```

Copy

```

        current_candle_copy = None
        should_trade_infer = True

    else:
        candle = self.current_candle[symbol]
        candle["close"] = snap.price
        candle["high"] = max(candle["high"], snap.price)
        candle["low"] = snap.price if candle["low"] == 0.0 else min(candle["low"],
snap.price)

        candle["volume"] = snap.volume
        candle["fut_cvd"] = snap.fut_cvd
        candle["liq_long"] = abs(snap.liq_long)
        candle["liq_short"] = abs(snap.liq_short)
        candle["coins_bid"] = abs(snap.coins_bid)
        candle["coins_ask"] = abs(snap.coins_ask)
        candle["dollars_bid"] = abs(snap.dollars_bid)
        candle["dollars_ask"] = abs(snap.dollars_ask)
        candle["tk_buy_cnt"] = abs(snap.tk_buy_cnt)
        candle["tk_sell_cnt"] = abs(snap.tk_sell_cnt)
        candle["fp_poc"] = snap.fp_poc
        candle["oi"] = snap.oi
        candle["funding"] = snap.funding
        candle["ls_ratio"] = snap.ls_ratio
        candle["rsi"] = snap.rsi
        candle["whale_idx"] = snap.whale_idx

    if history_copy:
        self._run_inference_with_copied_history(
            symbol,
            snap.price,
            trade_tracker,
            history_copy,
            current_candle_copy,
            trigger_trade=should_trade_infer,
        )

    return snap

```

This removes heavy per-tick Pandas inference. The predictor still receives ticks to maintain current OHLC, but only builds features when a candle closes.

C4. Fix warmup stale-entry behavior

Your `set_history()` already sets `_last_predict_bar`, which is correct. Keep that. This prevents immediate stale trades on restart.

But add a log:

```

with self._lock:
    self.candles.history[symbol] = collections.deque(clean_list, maxlen=1200)
    if clean_list:
        self._last_predict_bar[symbol] = clean_list[-1]["open_time"]
        print(f"[UnifiedPredictor] {symbol} warmup loaded: {len(clean_list)} bars. Last
closed={clean_list[-1]['open_time']}")

```

C5. Fix feature-generation order

At the start of `_run_inference_with_copied_history()`, replace the current raw DataFrame construction with:

```

df = self._standardize_history_df(history)
if df.empty or len(df) < MIN_WARMUP_BARS:
    return

btc_ref = self._make_btc_ref(symbol, df)

df_std = generate_features_standard(df.copy(), btc_ref.copy())
df_vwap, _ = prep_vwap(df.copy(), btc_ref.copy())

if df_std.empty or df_vwap.empty:
    return

df_std = df_std.replace([np.inf, -np.inf], np.nan).ffill().bfill().fillna(0.0)
df_vwap = df_vwap.replace([np.inf, -np.inf], np.nan).ffill().bfill().fillna(0.0)

```

This is one of the most important parity fixes.

D. Engine1TradeTracker patches

D1. Do not reject tight ATR stops; widen them

Inside `trigger_entry()`, replace:

```

# Guard: reject if stop_pct < 0.3% live tick noise kills tighter stops
stop_pct = stop_dist / entry_price
if stop_pct < 0.0030:
    return

```

with:

```

# Live execution minimum stop. Widen instead of rejecting.
min_stop_pct = float(os.environ.get("MIN_LIVE_STOP_PCT", "0.003"))
min_stop_dist = entry_price * min_stop_pct

if stop_dist < min_stop_dist:
    rr = abs(tp - entry_price) / stop_dist if stop_dist > 0 else 3.0
    stop_dist = min_stop_dist
    tp_dist = stop_dist * rr

    if direction == 1:
        sl = entry_price - stop_dist
        tp = entry_price + tp_dist
    else:
        sl = entry_price + stop_dist
        tp = entry_price - tp_dist

```

Then replace:

```

if atr > 0 and stop_dist > 5.0 * atr:
    return

```

with:

```

# Do not let the ATR guard conflict with mandatory live stop widening.
max_stop_atr_mult = float(os.environ.get("MAX_STOP_ATR_MULT", "12.0"))
if atr > 0 and stop_dist > max_stop_atr_mult * atr:
    print(
        f"[RiskGovernor] Entry blocked. {symbol} {strategy}: "
        f"stop_dist={stop_dist:.8f} > {max_stop_atr_mult:.1f} ATR {atr:.8f}"
    )
    return

```

D2. Parse MT5 symbol map from env

In `Engine1TradeTracker.__init__()`, before `MT5Broker(...)`:

```
import json as _json

try:
    mt5_symbol_map = _json.loads(os.environ.get("MT5_SYMBOL_MAP", "{}") or "{}")
except Exception:
    mt5_symbol_map = {}
```

Then pass it:

```
self.mt5_broker = MT5Broker(
    dry_run=not MT5_LIVE,
    account_size=initial_capital,
    risk_pct=ENGINE_RISK_PCT,
    magic=unique_magic,
    symbol_map=mt5_symbol_map,
)
```

Example `.env`:

```
MT5_LIVE=0
EXECUTION_MODE=LIVE
ENGINE_RISK_PCT=0.004
MT5_SYMBOL_MAP=
{"BTCUSD":"BTCUSD.a","ETHUSD":"ETHUSD.a","XAUUSD":"XAUUSD.a","NATGASUSD":"NATGAS.a"}
```

E. mt5_broker.py patches

E1. Add success retcodes

Inside `MT5Broker` class:

```
_SUCCESS_RETCODES = {
    mt5.TRADE_RETCODE_DONE,
    getattr(mt5, "TRADE_RETCODE_PLACED", 10000),
    getattr(mt5, "TRADE_RETCODE_DONE_PARTIAL", 10010),
}
```

Replace result checks like:

```
if result is not None and result.retcodes == mt5.TRADE_RETCODE_DONE:
```

with:

```
if result is not None and result.retcodes in self._SUCCESS_RETCODES:
```

Also replace:

```
if result is None or result.retcodes != mt5.TRADE_RETCODE_DONE:
```

with:

```
if result is None or result.retcodes not in self._SUCCESS_RETCODES:
```

This fixes pending orders being falsely treated as failed.

E2. Improve symbol mapping

Replace `_map_symbol()` with:

```
def _map_symbol(self, binance_symbol):
    if binance_symbol in self.symbol_map:
        return self.symbol_map[binance_symbol]

    if binance_symbol in self._symbol_cache:
        return self._symbol_cache[binance_symbol]

    base_sym = binance_symbol.replace("USD", "USD")

    if not self.connected:
        return base_sym

    candidates = []
    for pattern in (f"*(base_sym)", f"*(base_sym)*", f"*(base_sym.lower())*"):
        syms = mt5.symbols_get(group=pattern)
        if syms:
            candidates.extend(list(syms))

    if not candidates:
        self._symbol_cache[binance_symbol] = base_sym
        return base_sym

    def score(s):
        name = s.name
        trade_allowed = 1 if getattr(s, "trade_mode", 0) != mt5.SYMBOL_TRADE_MODE_DISABLED
        else 0
        visible = 1 if getattr(s, "visible", False) else 0
        exact = 1 if name == base_sym else 0
        starts = 1 if name.startswith(base_sym) else 0
        return (trade_allowed, visible, exact, starts, -len(name))

    chosen = sorted(candidates, key=score, reverse=True)[0].name
    mt5.symbol_select(chosen, True)

    self._symbol_cache[binance_symbol] = chosen
    return chosen
```

E3. Normalize to tick size, not only digits

Replace `_normalize_price()` with:

```
def _normalize_price(self, sym_info, price: float) -> float:
    digits = int(sym_info.digits)
    tick = float(getattr(sym_info, "trade_tick_size", 0.0) or getattr(sym_info, "point",
    0.0) or 0.0)

    p = float(price)
    if tick > 0:
        p = round(p / tick) * tick

    return round(p, digits)
```

E4. Use BUY/SELL for order_calc_profit() even for pending orders

Replace:

```
loss_per_lot = self._loss_per_lot(mt5_sym, order_type, exec_price, mt5_sl, sym_info)
```

with:

```
calc_order_type = mt5.ORDER_TYPE_BUY if direction == 1 else mt5.ORDER_TYPE_SELL  
loss_per_lot = self._loss_per_lot(mt5_sym, calc_order_type, exec_price, mt5_sl, sym_info)
```

E5. Accept pending order check retcodes

Replace:

```
if check is not None and check.retcodes not in (0, mt5.TRADE_RETCODE_DONE):
```

with:

```
if check is not None and check.retcodes not in (0, *self._SUCCESS_RETCODES):
```

If your Python version rejects starred set unpacking inside tuple, use:

```
valid_check_codes = (0, *self._SUCCESS_RETCODES)  
if check is not None and check.retcodes not in valid_check_codes:  
    ...
```

3. Trade Execution Gate Audit

Gate 1 — Warmup

Current risk:

- combined_seed_history.xlsx can exist but have bad timestamps or zero-filled columns.
- UnifiedLivePredictor requires >=250 bars.
- Some strategies effectively need 800+ bars depending on rolling features.

Fix:

- Load seed history before live.
- Normalize all timestamps to seconds.
- Log warmup per symbol.

Required runtime log:

```
[UnifiedPredictor] BTCUSD warmup loaded: 850 bars. Last closed=...
```

If any symbol says <250, it is not tradable.

Gate 2 — Strategy trigger

Major issues in current live_unified_predictor.py:

- Features are built before all required columns are standardized.
- rsi vs RSI mismatch.
- POC Price added too late.
- Bid USD, Ask USD, Delta USD added too late.
- NaNs can silently make comparisons false.

Required fix:

- Use _standardize_history_df() before all feature generation.
- Sanitize np.inf, -np.inf, and NaN.

Gate 3 — Probability threshold

Risk:

```
prob_threshold = m_data.get("prob_threshold", 0.6)
```

This fails if manifest is nested or has side-specific thresholds.

Fix:

```
m_data = self._manifest_for(strat, symbol)  
threshold = self._threshold_for(m_data, direction)
```

Also choose best candidate by edge, not long-first:

```
best = max(candidates, key=lambda x: (x["edge"], x["prob"]))
```

Gate 4 — Governor

Major silent blockers:

1. stop_pct < 0.003 rejects many trades.
2. stop_dist > 5 * ATR conflicts with the minimum stop.
- 3.Cooldowns can block after stale active trades.
4. Active dry-run trades from old logs can permanently block symbols.

Fix:

- Widen minimum stop instead of rejecting.
- Increase ATR guard to 12x or make it strategy-specific.
- Add structured logs for every rejection.
- Clear stale dry-run active trades older than 48h.

Gate 5 — Broker bridge

Major blockers:

1. MTS_LIVE = False hardcoded.
2. Missing explicit symbol map.
3. Pending orders treated as failure.
4. order_calc_profit() using pending order types.
5. Order types not explicitly mapped to mt5.

5. Broker filling mode may reject IOC.

Minimum required .env:

```
EXECUTION_MODE=LIVE
MT5_LIVE=0
ENGINE_RISK_PCT=0.004
MT5_SYMBOL_MAP={"BTCUSD": "BTCUSD.a", "ETHUSD": "ETHUSD.a", "XAUUSD": "XAUUSD.a"}
```

Set MT5_LIVE=1 only after dry-run verification.

4. OOS Backtest vs Live Parity Risks

I cannot verify your opt_s1_colab_standalone.py through opt_s6_colab_standalone.py because their contents were not included, but from the live code alone these are the high-risk parity breaks:

Area	Current live risk
Feature columns	Live adds several training columns after feature generation
EMA	Possible adjust=True vs adjust=False mismatch
VWAP / POC	POC Price may be missing when prep_vwap() runs
Entry fill	Live uses first tick/current price after bar close, OOS likely uses next bar open
Fees	Live uses 0.05% * 2; OOS may use different fee/slippage
Warmup	Live starts at 250 bars; some rolling features may require 800+
Direction	Unified model uses synthetic direction column, must match training exactly
Liquidation signs	S1 long-liq mapping must match OOS reversal/breakout logic

Recommended parity harness:

```
def assert_live_oos_parity(symbol: str, rows: list[dict], oos_feature_fn):
    predictor = UnifiedLivePredictor([symbol])
    live_df = predictor._standardize_history_df(rows)
    btc_ref = predictor._make_btc_ref(symbol, live_df)

    live_std = generate_features_standard(live_df.copy(), btc_ref.copy())
    oos_std = oos_feature_fn(live_df.copy(), btc_ref.copy())

    common = sorted(set(live_std.columns) & set(oos_std.columns))
    live_last = live_std[common].replace([np.inf, -np.inf], np.nan).fillna(0).iloc[-1]
    oos_last = oos_std[common].replace([np.inf, -np.inf], np.nan).fillna(0).iloc[-1]

    diff = (live_last - oos_last).abs().sort_values(ascending=False)
    print(diff.head(30))

    assert diff.head(1).iloc[0] < 1e-8, "LIVE/OOS feature parity failed"
```

Best long-term fix: **one shared feature_factory.py imported by both live and OOS**, not duplicated feature code.

5. Verification Checklist

Dry-run verification

1. Start with:

```
MT5_LIVE=0
EXECUTION_MODE=LIVE
ML_THREADS=2
MIN_LIVE_STOP_PCT=0.003
MAX_STOP_ATR_MULT=12
```

1. Run:

```
python Engine_1.py --skip-seed
```

1. Confirm:

```
[UnifiedPredictor] BTCUSD warmup loaded: >=250 bars
[UnifiedPredictor] Strategy 'S1_Liquidation': N/18 active ensemble models loaded
```

- 1. Confirm no latency warnings above 100ms for 30 minutes.
- 2. Confirm snapshot_debug.json contains nonzero:

```
"oi"
"funding"
"ls_ratio"
"fut_cvd"
"liq_long"
"liq_short"
```

1. Force a dry-run trade manually:

```
trade_tracker.trigger_entry(
    "BTCUSD",
    "TEST_GATE",
    1,
    100000.0,
    99700.0,
    101500.0,
    300.0,
    macro=0,
    vol_regime=0,
)
```

Expected:

```
[MT5 SIMULATION] Executing paper trade...
[EngineTradeTracker] MT5 successfully entered BTCUSD...
```

Broker mapping verification

Run separately:

```
from mt5_broker import MT5Broker
import json, os

symbol_map = json.loads(os.environ.get("MT5_SYMBOL_MAP", "{}") or "{}")

b = MT5Broker(dry_run=False, symbol_map=symbol_map)
assert b.connect()
```

```
for s in ["BTCUSD", "ETHUSD", "XAUUSD", "NATGASUSD"]:  
    mt5_sym = b_map_symbol(s)  
    px_buy = b.get_mt5_price(s, 1)  
    px_sell = b.get_mt5_price(s, -1)  
    print(s, "->", mt5_sym, px_buy, px_sell)
```

Every symbol must return a nonzero bid/ask.

Live-mode final checklist

Before MT5_LIVE=1:

- No Chrome reinjection loop for 2+ hours.
- No event-loop block >100ms for 2+ hours.
- Every tradable symbol has >=250 warmup bars.
- All 6 strategies show loaded model counts.
- pipeline_diag shows strategy triggers, probability values, and skip reasons.
- Dry-run trades appear in Engine_1_trade_logs.json.
- Broker symbol map confirmed.
- Minimum lot risk confirmed not above your allowed risk.

Then set:

MT5_LIVE=1

Copy

and run with smallest risk first.

📄 🍏 🗨️ ...

Your credit balance is negative. This run was stopped. Please top up to continue. [upgrade](#)

AI Chat

Stop paying for AI five times ->

Ask anything, create anything

+ GPT-5.5 Pro

🗨️ ↩